

AWS DEVOPS INTERVIEW GUIDE

Scenario 1: Auto Scaling Group Is Not Launching New Instances

Interview Question

Your application is experiencing high CPU usage, but the Auto Scaling Group is not launching new EC2 instances. How would you troubleshoot it?

Answer

If the Auto Scaling Group isn't scaling out, I verify the scaling policy, CloudWatch alarms, and ASG configuration.

Step 1: Check Auto Scaling Activity

Review the **Activity History** to identify why new instances were not launched.

Look for errors such as:

- Launch failed
 - Capacity limit reached
 - Launch Template error
-

Step 2: Verify CloudWatch Alarm

Ensure the scaling policy is linked to the correct CloudWatch alarm.

Example:

- CPU Utilization > 70%
 - Alarm state = **ALARM**
-

Step 3: Check ASG Limits

Verify:

- Desired Capacity
- Minimum Capacity
- Maximum Capacity

If **Desired** = **Maximum**, no additional instances will be launched.

Step 4: Verify Launch Template

Ensure the Launch Template has:

- Valid AMI
 - Instance Type
 - Security Group
 - IAM Role
 - Key Pair (if required)
-

Step 5: Check EC2 Limits

Verify the AWS account has sufficient EC2 quota for the required instance type.

Real-Time Scenario

A production application experienced high CPU usage, but no new instances were launched. Investigation showed the ASG had **Max Capacity** = **2**, and both instances were already running. After increasing the maximum capacity to **5**, Auto Scaling launched new instances automatically.

Best Practices

- Set appropriate Min, Desired, and Max capacity.
 - Monitor CloudWatch alarms.
 - Keep Launch Templates updated.
 - Configure Auto Scaling across multiple Availability Zones.
-

Interview Summary

"If an Auto Scaling Group doesn't launch new instances, I first review the ASG Activity History, verify CloudWatch alarms and scaling policies, check the Min/Desired/Max capacity, validate the Launch Template, and ensure there are sufficient EC2 service quotas."

Scenario 2: Auto Scaling Group Keeps Launching and Terminating Instances

Interview Question

Your Auto Scaling Group continuously launches new EC2 instances, but they are terminated after a few minutes. How would you troubleshoot it?

Answer

If an ASG keeps replacing instances, I first identify why the instances are being marked as unhealthy.

Step 1: Check ASG Activity History

Review the **Activity History**.

Look for messages like:

- ELB Health Check Failed
- EC2 Status Check Failed
- Instance terminated as unhealthy

Step 2: Check Target Group Health

Verify the EC2 instances are **Healthy** in the Target Group.

If they're unhealthy, check:

- Health check path
- Port
- Success code

Step 3: Verify the Application

SSH into the instance.

Check whether the application is running.

```
systemctl status myapp
```

Test the health endpoint.

```
curl localhost:8080/health
```

Step 4: Review Launch Template

Verify:

- Correct AMI
- User Data script
- IAM Role
- Security Group

A failed User Data script may prevent the application from starting.

Step 5: Check Logs

Review:

- Cloud-init logs
- Application logs
- System logs

Look for startup failures or configuration errors.

Real-Time Scenario

An Auto Scaling Group continuously terminated new instances after deployment. Investigation showed the **User Data script failed to install and start the application**, causing ALB health checks to fail. After fixing the script and updating the Launch Template, new instances became healthy and remained in service.

Best Practices

- Test Launch Templates before production.
 - Validate User Data scripts.
 - Use a dedicated `/health` endpoint.
 - Monitor ASG Activity History and Target Group health.
-

Interview Summary

"If an Auto Scaling Group continuously launches and terminates instances, I first review the ASG Activity History, verify Target Group health checks, ensure the application is running, validate the Launch Template and User Data script, and review system logs. In most cases, failed health checks or startup script errors are the root cause."

💡 Interview Tip

Common Follow-up Question:

What are the different health checks used by an Auto Scaling Group?

Answer:

- **EC2 Health Checks** – Detect hardware or OS failures.
- **ELB/ALB Health Checks** – Verify the application is responding.
- **Custom Health Checks** – Application-specific checks using CloudWatch or custom monitoring.

Using **ELB Health Checks** ensures instances are replaced only when the application is unhealthy, not just when the EC2 instance is running.

Scenario 3: Auto Scaling Group Does Not Replace Unhealthy Instances

Interview Question

One of your EC2 instances is unhealthy, but the Auto Scaling Group is not replacing it. How would you troubleshoot it?

Answer

If ASG is not replacing an unhealthy instance, I verify the health check configuration and ASG settings.

Step 1: Check Health Check Type

Verify the ASG health check type.

- **EC2** – Checks only instance health.
- **ELB/ALB** – Checks application health.

If only EC2 health checks are enabled, ASG won't replace an instance whose application has failed but whose EC2 instance is still running.

Step 2: Check Health Check Grace Period

Verify the **Health Check Grace Period**.

If it's too long, ASG waits before marking the instance unhealthy.

Step 3: Check Instance Health

Review the instance status in:

- Auto Scaling Group
- Target Group (if using ALB)

Confirm whether the instance is marked **Healthy** or **Unhealthy**.

Step 4: Verify Desired Capacity

Ensure the ASG has enough capacity to launch a replacement instance.

Check:

- Minimum Capacity
 - Desired Capacity
 - Maximum Capacity
-

Step 5: Review Activity History

Check the ASG **Activity History** for errors related to health checks or instance replacement.

Real-Time Scenario

A production application stopped responding, but the EC2 instance was still running. The ASG was using **EC2 health checks only**, so it considered the instance healthy. After changing the health check type to **ELB**, the unhealthy instance was terminated and replaced automatically.

Best Practices

- Use **ELB Health Checks** for web applications.
 - Configure an appropriate health check grace period.
 - Monitor HealthyHostCount and ASG Activity History.
 - Test health checks after every deployment.
-

Interview Summary

"If Auto Scaling doesn't replace an unhealthy instance, I first verify the health check type, ensure ELB health checks are enabled for application-level monitoring, review the health check grace period, check the instance health status, and analyze the ASG Activity History to identify why replacement wasn't triggered."

💡 Interview Tip

Common Follow-up Question:

Why should you use ELB health checks instead of only EC2 health checks?

Answer:

- **EC2 Health Check** → Verifies the VM and operating system are running.
- **ELB Health Check** → Verifies the application is actually responding.

Example:

- EC2 is running ✓
- Application crashed ✗

With only EC2 health checks, ASG won't replace the instance. With ELB health checks, it will automatically replace it.

Scenario 4: Auto Scaling Group Is Not Scaling In

Interview Question

Traffic has reduced, but the Auto Scaling Group is not terminating EC2 instances. How would you troubleshoot it?

Answer

If the ASG is not scaling in, I verify the scaling policy, CloudWatch alarms, and instance protection settings.

Step 1: Check CloudWatch Alarm

Verify the **scale-in** alarm is configured correctly.

Example:

- CPU Utilization < **30%**
- Alarm state = **ALARM**

If the alarm never enters the ALARM state, scale-in won't occur.

Step 2: Verify Scaling Policy

Ensure a **scale-in policy** is attached to the ASG.

Sometimes only a scale-out policy is configured.

Step 3: Check Instance Scale-In Protection

Verify whether **Scale-In Protection** is enabled.

If enabled, ASG will not terminate protected instances.

Step 4: Check Desired Capacity

Verify:

- Desired Capacity
- Minimum Capacity

If **Desired Capacity** = **Minimum Capacity**, ASG cannot terminate more instances.

Step 5: Review Activity History

Check the ASG Activity History for messages such as:

- Scale-in blocked
 - Instance protected from scale-in
 - Cooldown in progress
-

Real-Time Scenario

A production application had low traffic overnight, but five EC2 instances continued running. Investigation showed **Scale-In Protection** had been enabled during maintenance and was never removed. After disabling it, Auto Scaling terminated the extra instances and reduced costs.

Best Practices

- Configure both **scale-out** and **scale-in** policies.
 - Use **Target Tracking** for automatic scaling.
 - Enable Scale-In Protection only for critical instances.
 - Monitor ASG activities using CloudWatch.
-

Interview Summary

"If an Auto Scaling Group doesn't scale in, I verify the CloudWatch scale-in alarm, ensure a scale-in policy exists, check whether instances have Scale-In Protection enabled, review the Desired and Minimum capacity, and analyze the ASG Activity History for blocked scale-in events."

💡 Interview Tip

Common Follow-up Question:

What is Scale-In Protection?

Answer:

Scale-In Protection prevents specific EC2 instances from being terminated during a scale-in event.

Common use cases:

- Long-running batch jobs
- Deployment servers
- Critical application instances

It protects only from **Auto Scaling termination**, not from manual termination by an administrator.

Scenario 5: New EC2 Instances Launch Successfully but Fail After Scale-Out

Interview Question

During a traffic spike, Auto Scaling launches new EC2 instances, but they never become healthy and do not receive traffic. How would you troubleshoot it?

Answer

If new instances launch but don't serve traffic, I verify the application, Launch Template, Target Group, and health checks.

Step 1: Check Target Group Health

Verify the new instances are registered and check why they are **Unhealthy**.

Common reasons:

- Health check timeout
- Wrong port
- Wrong health check path

Step 2: Verify the Application

SSH into the new instance.

Check if the application is running.

```
systemctl status myapp
```

Test the health endpoint.

```
curl localhost:8080/health
```

Step 3: Check the Launch Template

Verify the Launch Template contains:

- Correct AMI
- User Data script
- IAM Role
- Security Group

If the User Data script fails, the application may never start.

Step 4: Review Logs

Check:

- Cloud-init logs
- Application logs
- System logs

Look for startup failures or dependency errors.

Step 5: Verify Security Groups

Ensure:

- ALB Security Group can reach the EC2 Security Group.
 - Required application ports are open.
-

Real-Time Scenario

During a production traffic spike, Auto Scaling launched new instances, but they never became healthy. Investigation showed the **latest AMI didn't include the application service**. A corrected AMI was created, the Launch Template was updated, and newly launched instances immediately passed the health checks.

Best Practices

- Test AMIs before updating the Launch Template.
- Validate User Data scripts.
- Keep health checks lightweight.

- Automate application startup during boot.
-

Interview Summary

"If newly launched Auto Scaling instances don't become healthy, I first check the Target Group health, verify the application is running, validate the Launch Template and User Data script, review startup logs, and ensure the Security Groups allow traffic from the Load Balancer. In production, outdated AMIs and failed User Data scripts are the most common causes."

💡 Interview Tip

Common Follow-up Question:

What are the most common reasons new Auto Scaling instances fail?

Answer:

- Incorrect AMI
 - Failed User Data script
 - Application not started
 - Wrong Target Group port
 - Health check path mismatch
 - Security Group misconfiguration
 - Missing IAM Role
 - Dependency failures (database, Secrets Manager, etc.)
-

Scenario 6: Scheduled Scaling Is Not Working

Interview Question

You configured Scheduled Scaling to increase EC2 instances before business hours, but the Auto Scaling Group didn't scale. How would you troubleshoot it?

Answer

If Scheduled Scaling isn't working, I verify the scheduled action, time zone, and ASG capacity settings.

Step 1: Check Scheduled Action

Verify the scheduled action:

- Is enabled
 - Has the correct date and time
 - Is associated with the correct Auto Scaling Group
-

Step 2: Verify Time Zone

Check whether the scheduled action is configured in the correct **time zone (UTC)**.

A timezone mismatch is a common cause of scaling at the wrong time.

Step 3: Check ASG Capacity

Verify:

- Minimum Capacity
- Desired Capacity
- Maximum Capacity

If **Desired Capacity** is already equal to **Maximum Capacity**, no new instances will be launched.

Step 4: Review Activity History

Check the ASG **Activity History** for:

- Scheduled action executed
 - Launch failures
 - Capacity limit reached
-

Step 5: Verify Launch Template

If the scheduled action executed but instances weren't created, validate:

- AMI
- Instance Type
- IAM Role
- EC2 Service Quotas

Real-Time Scenario

An application was configured to scale from **2 to 6 instances every weekday at 8 AM**. Users experienced slow performance because the scheduled action was configured for **8 AM UTC** instead of **8 AM local time**. After correcting the schedule, the instances launched before business hours and the application handled the morning traffic without issues.

Best Practices

- Use **UTC** when configuring scheduled actions.
 - Test scheduled scaling in a non-production environment.
 - Monitor Auto Scaling Activity History.
 - Use **Target Tracking** for unpredictable traffic and **Scheduled Scaling** for predictable traffic.
-

Interview Summary

"If Scheduled Scaling doesn't work, I verify the scheduled action, confirm the correct UTC schedule, check the Auto Scaling Group's Min, Desired, and Max capacity, review the Activity History, and validate the Launch Template and service quotas if instances fail to launch."

💡 Interview Tip

Common Follow-up Question:

When do you use Scheduled Scaling instead of Target Tracking?

Use Scheduled Scaling when traffic is predictable, such as:

- Office hours (9 AM–6 PM)
- Weekend traffic
- Flash sales
- Batch jobs
- Monthly payroll processing

Use Target Tracking when traffic is unpredictable, such as:

- E-commerce websites
- Public APIs

- Customer-facing web applications
-

Scenario 7: Target Tracking Scaling Policy Is Not Triggering

Interview Question

Your Auto Scaling Group uses a Target Tracking policy based on CPU utilization, but no new instances are launched even though CPU usage is high. How would you troubleshoot it?

Answer

If Target Tracking isn't triggering, I verify the scaling policy, CloudWatch metrics, and ASG configuration.

Step 1: Check the Target Tracking Policy

Verify the policy is attached to the correct Auto Scaling Group.

Example:

- Metric: **Average CPU Utilization**
 - Target Value: **70%**
-

Step 2: Check CloudWatch Metrics

Review:

- CPUUtilization
- NetworkIn/Out
- RequestCount (if applicable)

Ensure the metric has actually crossed the target threshold.

Step 3: Verify ASG Capacity

Check:

- Desired Capacity

- Maximum Capacity

If the ASG is already at **Maximum Capacity**, it cannot launch more instances.

Step 4: Check Instance Warm-Up

Verify the **Instance Warm-Up** period.

If it's too long, Auto Scaling waits before evaluating the next scaling action.

Step 5: Review Activity History

Check the ASG Activity History for:

- Scaling policy execution
 - Launch failures
 - Cooldown or warm-up delays
-

Real-Time Scenario

A production application reached **90% CPU utilization**, but Auto Scaling didn't launch new instances. Investigation showed the ASG had already reached its **Maximum Capacity of 3 instances**. After increasing the maximum capacity to 6, Target Tracking successfully launched additional instances and reduced CPU usage.

Best Practices

- Configure realistic target values (e.g., 60–70% CPU).
 - Set an appropriate instance warm-up time.
 - Monitor CloudWatch metrics and ASG Activity History.
 - Ensure Maximum Capacity supports expected traffic spikes.
-

Interview Summary

"If a Target Tracking policy isn't triggering, I verify the policy configuration, review CloudWatch metrics to ensure the threshold is exceeded, check the Auto Scaling Group's Maximum Capacity, verify the instance warm-up period, and review the Activity History for any scaling errors or delays."

💡 Interview Tip

Common Follow-up Question:

What are the different Auto Scaling policy types?

Policy	Use Case
Target Tracking	Maintain a target metric (e.g., 60% CPU)
Simple Scaling	Add or remove a fixed number of instances
Step Scaling	Scale differently based on the severity of the metric
Scheduled Scaling	Scale at predefined dates and times

Target Tracking is the most commonly used policy because it automatically adjusts capacity to maintain the desired metric.

Scenario 8: Availability Zone Failure and Auto Scaling Recovery

Interview Question

One Availability Zone (AZ) goes down, and several EC2 instances become unavailable. How does Auto Scaling handle this, and how would you troubleshoot it?

Answer

If an Availability Zone fails, Auto Scaling should automatically maintain the desired capacity by launching instances in the remaining healthy AZs.

Step 1: Verify the AZ Failure

Check:

- EC2 Status Checks
- AWS Health Dashboard
- CloudWatch Alarms

Confirm the issue is limited to one Availability Zone.

Step 2: Check Auto Scaling Configuration

Verify the ASG is configured to use **multiple Availability Zones**.

If it's configured for only one AZ, Auto Scaling cannot recover automatically.

Step 3: Verify Desired Capacity

Ensure:

- Desired Capacity is maintained.
- ASG is launching replacement instances in the healthy AZ.

Review the **Activity History** for launch events.

Step 4: Check Load Balancer

Verify the **ALB/NLB** is routing traffic only to healthy instances.

Ensure the new instances pass the health checks before receiving traffic.

Step 5: Verify Capacity & Quotas

If new instances are not launching, check:

- EC2 Service Quotas
 - Subnet IP availability
 - Launch Template configuration
-

Real-Time Scenario

An AWS Availability Zone experienced an outage, causing three application servers to become unavailable. Since the Auto Scaling Group was configured across **three Availability Zones**, it automatically launched replacement instances in the remaining healthy AZs. The ALB routed traffic only to healthy instances, and users experienced minimal downtime.

Best Practices

- Deploy Auto Scaling Groups across **at least two Availability Zones**.
 - Use ALB/NLB with health checks.
 - Monitor CloudWatch alarms and AWS Health events.
 - Keep sufficient subnet IPs available in each AZ.
-

Interview Summary

"If an Availability Zone fails, I first verify the outage using AWS Health and CloudWatch. Then I ensure the Auto Scaling Group spans multiple AZs, verify replacement instances are launched, confirm the Load Balancer routes traffic only to healthy instances, and check service quotas or subnet capacity if replacement instances fail to launch."

💡 Interview Tip

Common Follow-up Question:

Can Auto Scaling recover from an Availability Zone failure if it's configured in only one AZ?

Answer:

No.

If the ASG is configured in only one Availability Zone and that AZ fails, Auto Scaling cannot launch replacement instances.

Production Best Practice:

- Deploy ASGs across **multiple Availability Zones**.
 - Place the ALB/NLB across the same AZs.
 - Use Multi-AZ databases to avoid a single point of failure.
-

Scenario 9: Zero-Downtime Deployment Using Auto Scaling

Interview Question

How would you deploy a new application version using Auto Scaling without causing downtime?

Answer

For zero-downtime deployments, I use **Auto Scaling + ALB + Launch Templates** so that users continue to access healthy instances while the new version is deployed.

Step 1: Create a New Launch Template Version

Update the Launch Template with:

- New AMI (or application version)
 - Updated User Data (if required)
 - New configuration
-

Step 2: Update the Auto Scaling Group

Associate the ASG with the new Launch Template version.

New instances will be launched using the updated configuration.

Step 3: Launch New Instances

Increase the **Desired Capacity** or perform an **Instance Refresh**.

Wait until the new instances:

- Start successfully
 - Pass ALB health checks
-

Step 4: Remove Old Instances

Once the new instances are healthy, Auto Scaling gradually terminates the old instances.

This ensures users always have healthy instances serving requests.

Step 5: Monitor and Roll Back

Monitor:

- CloudWatch Metrics
- ALB Target Health
- Application Logs

If issues occur, roll back to the previous Launch Template version.

Real-Time Scenario

A production application required a new release without downtime. A new Launch Template version was created, and an **Instance Refresh** was started. Auto Scaling launched new instances, waited for them to pass ALB health checks, and then terminated the old instances one by one. Users experienced **zero downtime** throughout the deployment.

Best Practices

- Use **Instance Refresh** for EC2 Auto Scaling deployments.
 - Keep Launch Templates version-controlled.
 - Never terminate old instances until new ones are healthy.
 - Always have a rollback Launch Template available.
-

Interview Summary

"For zero-downtime deployments, I create a new Launch Template version, update the Auto Scaling Group, launch new instances using Instance Refresh, wait for them to pass ALB health checks, and then gradually terminate the old instances. If any issue occurs, I roll back to the previous Launch Template version."

Interview Tip

Common Follow-up Question:

What is Instance Refresh?

Answer:

Instance Refresh is an Auto Scaling feature that **gradually replaces existing EC2 instances with new ones** using the latest Launch Template or Launch Configuration while maintaining the desired capacity.

Benefits:

- Zero downtime
- Automatic rollback (if configured)
- Controlled replacement of instances

- Integrated with ALB health checks
-

Scenario 10: Design a Highly Available Auto Scaling Architecture

Interview Question

How would you design a highly available, scalable, and fault-tolerant Auto Scaling architecture for a production application?

Answer

For a production environment, I design the Auto Scaling architecture to ensure **high availability, automatic scaling, and fault tolerance**.

Step 1: Deploy Across Multiple Availability Zones

Configure the Auto Scaling Group across **at least two Availability Zones**.

This ensures the application remains available even if one AZ fails.

Step 2: Place an ALB in Front

Deploy an **Application Load Balancer (ALB)** to distribute traffic across healthy EC2 instances.

Only healthy instances should receive requests.

Step 3: Configure Auto Scaling Policies

Use:

- **Target Tracking** for dynamic scaling (e.g., CPU at 60–70%)
 - **Scheduled Scaling** for predictable traffic (business hours, sales events)
-

Step 4: Use Launch Templates

Store:

- AMI
- Instance Type
- IAM Role
- Security Groups
- User Data

Version the Launch Template to simplify deployments and rollbacks.

Step 5: Enable Monitoring

Monitor:

- CPU Utilization
- TargetResponseTime
- HealthyHostCount
- RequestCount

Configure CloudWatch alarms to trigger scaling actions.

Step 6: Plan for Recovery

- Enable **Instance Refresh** for zero-downtime deployments.
 - Use **ALB health checks** so unhealthy instances are replaced automatically.
 - Keep regular AMIs and EBS snapshots for disaster recovery.
-

Real-Time Scenario

A production e-commerce application experienced heavy traffic during a festival sale. The architecture consisted of an **ALB**, an **Auto Scaling Group** across **three Availability Zones**, and **Target Tracking** based on CPU utilization. Auto Scaling increased capacity from **4 to 12 instances** during peak traffic and scaled back down after the event, maintaining application availability while optimizing costs.

Best Practices

- Deploy ASGs across multiple AZs.
- Use ALB with health checks.
- Prefer Target Tracking over Simple Scaling.
- Use Instance Refresh for deployments.

- Monitor CloudWatch metrics and set alarms.
 - Regularly test scaling and recovery.
-

Interview Summary

"For a production Auto Scaling architecture, I deploy the ASG across multiple Availability Zones behind an ALB, use Target Tracking and Scheduled Scaling policies, manage infrastructure with Launch Templates, monitor CloudWatch metrics, and use Instance Refresh for zero-downtime deployments. This provides high availability, automatic scaling, and fault tolerance."

💡 Interview Tips

What metrics can trigger Auto Scaling?

- CPU Utilization
 - Request Count per Target (via ALB)
 - Network In/Out
 - Custom CloudWatch Metrics (e.g., queue length, application response time)
-

Which Scaling Policy should you choose?

Policy	Best Use Case
Target Tracking	Dynamic and unpredictable traffic
Step Scaling	Different scaling actions based on severity
Simple Scaling	Basic environments (less commonly used)
Scheduled Scaling	Predictable traffic patterns
